

Pontificia Universidad Javeriana

Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

Introducción a Sistemas Distribuidos

Taller de Evaluación de Rendimiento MPI

Integrantes:

Ana Sofía Arboleda Convers
Paula Gabriela
Daniel Felipe Ramírez Vargas
Guillermo Andrés Aponte Cárdenas
Samuel Pico Durán

Docente: John Jairo Corredor

Fecha: 20 de marzo de 2026

Índice

1. Objetivos	3
1.1. Objetivo General	3
1.2. Objetivos Específicos	3
2. Marco Teórico	3
2.1. Sistema de ficheros compartidos	3
2.2. Computación Paralela	3
2.3. MPI (Message Passing Interface)	3
2.4. OpenMP	3
2.5. Métricas de Rendimiento	3
3. Especificaciones de los Equipos	3
4. Procedimiento de Creación del Clúster	3
4.1. Obtención de Direcciones IP	3
4.2. Mapeo de Nombres a Direcciones IP (/etc/hosts)	4
4.3. Configuración de Llaves SSH sin Contraseña	5
4.3.1. Generación del par de llaves	5
4.3.2. Distribución de la llave pública	5
4.3.3. Verificación de la conexión	6
4.4. Sistema de Archivos Compartido (NFS)	6
4.4.1. Instalación de NFS en todas las máquinas	6
4.4.2. Creación del directorio compartido (solo en el Master)	7
4.4.3. Configuración de exportaciones NFS (solo en el Master)	7
4.4.4. Inicio del servicio NFS (solo en el Master)	8
4.4.5. Montaje del directorio compartido en los Workers	8
4.5. Instalación de OpenMPI	9
4.6. Configuración del Hostfile de MPI	9
4.7. Compilación de los Archivos Fuente	10
4.8. Ejecución del Programa	11
5. Diseño de Experimentos	11
5.1. Variables del Experimento	13
5.2. Constantes del Experimento	14
5.3. Criterio de Divisibilidad	15
5.4. Archivos Fuente del Experimento	15
5.4.1. moduloMPI.h — Declaraciones compartidas	16
5.4.2. moduloMPI.c — Funciones compartidas de soporte	16
5.4.3. mxmOmpMPIfx.c — Algoritmo Filas × Columnas	16
5.4.4. mxmOmpMPIfxt.c — Algoritmo Filas × Transpuesta	17
5.4.5. Makefile — Compilación consistente	17
5.4.6. lanzadorMPI.pl — Automatización del experimento	17

6. Análisis de Resultados	18
6.1. Tiempo Promedio de Ejecución	18
6.2. Speedup	18
6.3. Eficiencia	18
6.4. Overhead de Comunicación	18
6.5. Comparación entre Algoritmos FxC y FxT	18
6.6. Limitaciones del Experimento	18
7. Conclusiones	18

Resumen

1. Objetivos

1.1. Objetivo General

1.2. Objetivos Específicos

2. Marco Teórico

2.1. Sistema de ficheros compartidos

2.2. Computación Paralela

2.3. MPI (Message Passing Interface)

2.4. OpenMP

2.5. Métricas de Rendimiento

3. Especificaciones de los Equipos

4. Procedimiento de Creación del Clúster

El clúster se construyó sobre los equipos de la sala, conectados por red local, todos con Ubuntu Linux como sistema operativo. El proceso de configuración sigue el siguiente orden: primero se obtienen las direcciones IP de cada máquina y se registran en el archivo de hosts, luego se habilita la comunicación segura sin contraseña entre nodos mediante SSH, después se configura el sistema de archivos compartido con NFS, y finalmente se instala y configura OpenMPI. Cada uno de estos pasos se describe a continuación.

4.1. Obtención de Direcciones IP

Para que los nodos puedan comunicarse entre sí, es necesario conocer la dirección IP de cada máquina dentro de la red local. Se utilizó el siguiente comando en cada equipo:

```
ifconfig
```

Donde:

- `ifconfig` — (*interface configuration*) para consultar y configurar interfaces de red. Muestra información de cada interfaz activa: dirección IP, máscara de red, dirección MAC y estadísticas de tráfico.

La dirección relevante es la que aparece como **inet** en la interfaz de red local. Se registró la IP de cada equipo para el paso siguiente. Las IPs obtenidas fueron:

```
10.195.23.70    master
10.195.23.160   nodo0
10.195.23.161   nodo1
10.195.23.39    nodo2
```

4.2. Mapeo de Nombres a Direcciones IP (/etc/hosts)

Se utilizó el archivo `/etc/hosts` como una tabla local de resolución de nombres: antes de consultar un servidor, el sistema busca aquí si el nombre solicitado tiene una IP asignada. Agregar todos los nodos del clúster en este archivo permite que cada máquina reconozca a las demás por su nombre en lugar de por IP.

Importante: Este proceso debe ejecutarse en *todas* las máquinas del clúster, incluyendo el Master y nodos.

Se edita el archivo con el siguiente comando:

```
sudo nano /etc/hosts
```

Donde:

- **nano** — editor de texto en terminal, sencillo en Ubuntu.
- **/etc/hosts** — archivo del sistema que mapea nombres a IPs; solo puede modificarlo el administrador (**sudo**).

Dentro del archivo, al final del contenido existente, se agregan las líneas con la IP y el nombre asignado a cada nodo. Los nombres elegidos fueron **master** para el nodo coordinador y **nodo0**, **nodo1**, **nodo2** para los Workers:

```
10.195.23.70    master
10.195.23.160   nodo0
10.195.23.161   nodo1
10.195.23.39    nodo2
```

Para guardar en **nano**: **Ctrl+O**, luego **Enter** para confirmar, y **Ctrl+X** para salir.

Para verificar que la resolución de nombres funciona correctamente, se utiliza el comando **ping** hacia cada nodo:

```
ping nodo0 -c 4
```

Donde:

- **ping** — envía paquetes ICMP de prueba a una dirección o nombre de host para verificar conectividad.
- **nodo0** — nombre del nodo destino (resuelto gracias a `/etc/hosts`).
- **-c 4** — limita el número de paquetes enviados a 4; sin esta opción, **ping** se ejecuta indefinidamente.

Una salida exitosa muestra `0% packet loss`, lo que confirma que la comunicación entre nodos funciona y que el mapeo de nombres es correcto.

4.3. Configuración de Llaves SSH sin Contraseña

MPI utiliza SSH (*Secure Shell*) para lanzar procesos de forma remota en los nodos Workers desde el Master. SSH es un protocolo de comunicación cifrada que permite controlar otra máquina a través de la red de forma segura. Para que MPI funcione de manera automática y sin interrupciones, SSH no debe solicitar contraseña en ninguna conexión entre nodos.

La solución es usar *autenticación por llave pública*: cada máquina genera un par de llaves criptográficas (una privada que nunca sale del equipo y una pública que se distribuye a los demás nodos). Cuando un nodo intenta conectarse a otro, ambos verifican sus llaves y, si coinciden, la conexión se establece sin necesidad de contraseña.

Importante: Este proceso debe realizarse en *todas* las máquinas. Además, cada nodo debe poder conectarse a sí mismo y a todos los demás sin contraseña, incluyendo el Master conectándose a sí mismo (`ssh master`).

4.3.1. Generación del par de llaves

En **cada máquina** se genera el par de llaves con:

```
ssh-keygen -t rsa -b 2048
```

Donde:

- `ssh-keygen` — herramienta para generar, gestionar y convertir llaves de autenticación SSH.
- `-t rsa` — especifica el tipo de algoritmo criptográfico; RSA es el estándar más compatible.
- `-b 2048` — define el tamaño de la llave en bits; 2048 bits ofrece un balance adecuado entre seguridad y rendimiento.

El comando solicita la ruta donde guardar las llaves (presionar **Enter** para aceptar la ruta por defecto `~/.ssh/id_rsa`) y una contraseña de protección (presionar **Enter** dos veces para dejarlo vacío, lo cual es necesario para conexiones automáticas sin intervención manual).

Esto genera dos archivos en `~/.ssh/`:

- `id_rsa` — llave **privada**. Nunca debe compartirse ni salir del equipo.
- `id_rsa.pub` — llave **pública**. Es la que se copia a los demás nodos.

4.3.2. Distribución de la llave pública

Desde **cada máquina**, se copia la llave pública a todos los demás nodos (incluyendo a sí mismo):

```
ssh-copy-id master
ssh-copy-id nodo0
ssh-copy-id nodo1
ssh-copy-id nodo2
```

Donde:

- `ssh-copy-id` — copia la llave pública del equipo local al archivo `~/.ssh/authorized_keys` del nodo destino. SSH verifica este archivo para permitir conexiones sin contraseña.
- `master, nodo0...` — nombre del nodo destino, resuelto gracias al archivo `/etc/hosts` configurado anteriormente.

Este comando pedirá la contraseña del usuario del nodo destino una única vez (la última vez que se pedirá).

4.3.3. Verificación de la conexión

Para confirmar que la autenticación sin contraseña funciona correctamente, se intenta conectar a cada nodo:

```
ssh nodo0
```

Si la conexión se establece directamente sin solicitar contraseña, el proceso fue exitoso. Para salir de la sesión remota:

```
exit
```

4.4. Sistema de Archivos Compartido (NFS)

NFS permite que todos los nodos del clúster accedan al mismo directorio como si fuera local en cada máquina. Esto es esencial para que los ejecutables compilados y los resultados de los experimentos estén disponibles en todos los nodos sin necesidad de copiarlos manualmente. La arquitectura es servidor-cliente: el Master exporta el directorio y los Workers lo montan.

4.4.1. Instalación de NFS en todas las máquinas

En **todas las máquinas** (Master y Workers):

```
sudo apt install -y nfs-common
```

Donde:

- `apt install` — gestor de paquetes de Ubuntu; descarga e instala software desde los repositorios oficiales.
- `-y` — responde “sí” automáticamente a todas las confirmaciones durante la instalación.

- **nfs-common** — paquete con las herramientas necesarias para montar sistemas de archivos NFS remotos.

Adicionalmente, en el **nodo Master** se instala el servidor NFS:

```
sudo apt install -y nfs-kernel-server
```

- **nfs-kernel-server** — implementación del servidor NFS en el kernel de Linux; permite exportar directorios a otros equipos.

4.4.2. Creación del directorio compartido (solo en el Master)

Se crea el directorio `/Almacen` que será exportado a todos los nodos:

```
sudo mkdir /Almacen  
sudo chown -R sistemas:sistemas /Almacen
```

Donde:

- **mkdir** — crea un nuevo directorio.
- **/Almacen** — ruta del directorio en la raíz del sistema de archivos; debe existir con la misma ruta en todos los nodos para garantizar coherencia de rutas al ejecutar MPI.
- **chown** — cambia el propietario de un archivo o directorio (*change owner*).
- **-R** — aplica el cambio de forma recursiva a todos los archivos y subdirectorios dentro.
- **sistemas:sistemas** — nombre del usuario y grupo del sistema; se asigna como propietario para poder leer y escribir en el directorio sin necesidad de **sudo**.

4.4.3. Configuración de exportaciones NFS (solo en el Master)

Se edita el archivo `/etc/exports`, que define qué directorios exporta el servidor NFS y a quién tiene acceso:

```
sudo nano /etc/exports
```

Se agrega una línea por cada nodo Worker con la IP real del equipo:

```
/Almacen 10.195.23.160(rw,no_root_squash,subtree_check,fsid=0)  
/Almacen 10.195.23.161(rw,no_root_squash,subtree_check,fsid=0)  
/Almacen 10.195.23.39(rw,no_root_squash,subtree_check,fsid=0)
```

El significado de cada opción entre paréntesis es:

- **rw** — (*read-write*) el cliente puede leer y escribir en el directorio exportado.
- **no_root_squash** — permite que el usuario **root** del cliente opere con privilegios de root en el directorio exportado. Sin esta opción, root del cliente se mapea a un usuario sin privilegios, lo que puede causar errores de permiso al compilar o ejecutar.

- `subtree_check` — el servidor verifica que el archivo solicitado pertenece efectivamente al directorio exportado, añadiendo una capa de seguridad.
- `fsid=0` — asigna un identificador único al sistema de archivos exportado, necesario para que NFS lo identifique correctamente.

Para aplicar los cambios y exportar el directorio sin reiniciar el servicio:

```
sudo exportfs -rv
```

Donde:

- `exportfs` — herramienta para gestionar las exportaciones NFS activas.
- `-r` — re-exporta todos los directorios listados en `/etc/exports`.
- `-v` — modo verbose; muestra en consola qué directorios se están exportando y a qué clientes.

4.4.4. Inicio del servicio NFS (solo en el Master)

Se habilita e inicia el servidor NFS:

```
sudo systemctl enable nfs-kernel-server
sudo systemctl start nfs-kernel-server
sudo systemctl status nfs-kernel-server
```

Donde:

- `systemctl` — herramienta para gestionar servicios del sistema en Linux (iniciar, detener, habilitar, verificar estado).
- `enable` — configura el servicio para que inicie automáticamente al arrancar el sistema.
- `start` — inicia el servicio de forma inmediata.
- `status` — muestra el estado actual del servicio. Una salida exitosa muestra `active (running)`.

4.4.5. Montaje del directorio compartido en los Workers

En **cada nodo Worker**, se crea el punto de montaje local y se monta el directorio exportado por el Master:

```
sudo mkdir /Almacen
sudo mount master:/Almacen /Almacen
```

Donde:

- `mount` — comando que conecta (monta) un sistema de archivos remoto en un punto del árbol de directorios local.
- `master:/Almacen` — especifica el servidor NFS (`master`) y el directorio que exporta.

- `/Almacen` — punto de montaje local en el Worker; a partir de este momento, todo lo que el Master escriba en `/Almacen` será visible inmediatamente en esta ruta en todos los Workers.

4.5. Instalación de OpenMPI

OpenMPI es la implementación de código abierto del estándar MPI utilizada en este laboratorio. Proporciona las herramientas necesarias para compilar y ejecutar programas paralelos distribuidos en C.

En **todas las máquinas** del clúster se ejecuta:

```
sudo apt install -y openmpi-bin openmpi-common libopenmpi-dev
libgtk2.0-dev
```

Los paquetes instalados son:

- `openmpi-bin` — contiene los ejecutables principales: `mpirun` (lanzador de procesos MPI), `mpicc` (compilador MPI para C) y `mpiexec`.
- `openmpi-common` — archivos de configuración compartidos y datos comunes necesarios para el funcionamiento de OpenMPI.
- `libopenmpi-dev` — librerías y archivos de cabecera (*headers*) necesarios para compilar código C que use MPI, incluyendo `mpi.h`.
- `libgtk2.0-dev` — librería de interfaces gráficas; requerida por algunas herramientas auxiliares de OpenMPI.

Para verificar que la instalación fue exitosa:

```
mpirun --version
```

La salida debe mostrar la versión instalada de OpenMPI, por ejemplo: `mpirun (Open MPI) 4.x.x`.

4.6. Configuración del Hostfile de MPI

El *hostfile* es un archivo de texto que le indica a MPI cuáles son los nodos disponibles en el clúster y cuántos procesos puede ejecutar cada uno simultáneamente (*slots*). MPI lo consulta al ejecutar `mpirun` para saber cómo distribuir los procesos.

El ZIP del profesor incluye un ejemplo llamado `filehost`. Se ubica en la carpeta compartida y se edita según la configuración del clúster:

```
nano /Almacen/filehost
```

```
master    slots=4
nodo0     slots=4
nodo1     slots=4
nodo2     slots=4
```

Donde:

- Cada línea representa un nodo del clúster identificado por su nombre (resuelto vía `/etc/hosts`).
- `slots=4` — indica cuántos procesos MPI puede alojar ese nodo de forma simultánea. Este valor debe corresponderse con el número de núcleos o hilos lógicos del procesador de cada máquina. Con 5 nodos de 4 slots cada uno, el máximo de procesos totales es $np = 5 \times 4 = 20$.

Nota sobre el proceso MASTER: En los algoritmos del ZIP, el proceso con `rank = 0` es el MASTER y **no realiza cálculos**; solo distribuye y recolecta datos. Por lo tanto, el número efectivo de Workers es $np - 1$. Esto implica que la dimensión de la matriz N debe ser divisible por $(np - 1)$, no por np .

4.7. Compilación de los Archivos Fuente

Los archivos fuente del experimento se ubican en la carpeta compartida para que todos los nodos accedan al mismo ejecutable:

```
cp -r /ruta/evalMxM_MPI/* /Almacen/
cd /Almacen
```

Donde:

- `cp -r` — copia archivos y directorios de forma recursiva.
- `cd` — cambia el directorio de trabajo actual (*change directory*).

La compilación se realiza desde el **nodo Master** con el **Makefile** incluido:

```
make
```

`make` lee el archivo **Makefile** y ejecuta los comandos de compilación definidos en él. En este caso compila los dos algoritmos:

```
mpicc -o mxmOmpMPIfxc mxmOmpMPIfxc.c moduloMPI.c -lm -fopenmp
mpicc -o mxmOmpMPIfxt mxmOmpMPIfxt.c moduloMPI.c -lm -fopenmp
```

El significado de cada parte del comando de compilación es:

- `mpicc` — compilador de MPI para C; es una capa sobre `gcc` que añade automáticamente las librerías y rutas necesarias para MPI.
- `-o mxmOmpMPIfxc` — nombre del ejecutable de salida.
- `mxmOmpMPIfxc.c moduloMPI.c` — archivos fuente a compilar; `moduloMPI.c` contiene las funciones compartidas por ambos algoritmos (transposición, inicialización de matrices, medición de tiempos, etc.).
- `-lm` — enlaza la librería matemática de C (`math.h`), necesaria para funciones como `sqrt`.
- `-fopenmp` — habilita el soporte de OpenMP, necesario para las directivas `#pragma omp` usadas en el cálculo paralelo dentro de cada Worker.

Para verificar que los ejecutables fueron generados:

```
ls -lh /Almacen/mxmOmpMPIfxc /Almacen/mxmOmpMPIfxt
```

4.8. Ejecución del Programa

La sintaxis general para ejecutar cualquiera de los dos algoritmos es:

```
mpirun -hostfile /Almacen/filehost -np <np> ./<ejecutable> <N>
      <nH>
```

Donde:

- `mpirun` — lanzador de procesos MPI; conecta a los nodos via SSH, inicia el número de procesos indicado y coordina la comunicación entre ellos.
- `-hostfile /compartido/filehost` — indica el archivo que lista los nodos disponibles y sus slots.
- `-np <np>` — número total de procesos MPI a lanzar, incluyendo el MASTER. Con `-np 4` se tendrán 1 MASTER y 3 Workers.
- `<ejecutable>` — `mxmOmpMPIfxc` (algoritmo Filas×Columnas) o `mxmOmpMPIfxt` (algoritmo Filas×Transpuesta).
- `<N>` — dimensión de la matriz cuadrada $N \times N$. Debe ser divisible por $(np - 1)$.
- `<nH>` — número de hilos OpenMP que usará cada Worker internamente para el cálculo. Típicamente igual al número de núcleos por máquina.

Ejemplo concreto con una matriz de 800×800 , 4 procesos MPI y 2 hilos OpenMP por Worker:

```
mpirun -hostfile /Almacen/filehost -np 4 ./mxmOmpMPIfxc 800 2
```

La salida imprime el tiempo de ejecución en microsegundos. Para verificar el funcionamiento con una matriz pequeña (el programa imprime las matrices si $N < 13$):

```
mpirun -hostfile /Almacen/filehost -np 4 ./mxmOmpMPIfxc 8 1
```

5. Diseño de Experimentos

Con el fin de evaluar de manera rigurosa el rendimiento de la multiplicación de matrices en un entorno distribuido MPI con paralelismo OpenMP, se diseñó un experimento controlado que varía de forma sistemática los factores que inciden en el tiempo de ejecución. El eje central del diseño experimental es el script de automatización `lanzadorMPI.pl`, escrito en Perl, el cual define las combinaciones de parámetros a ejecutar, orquesta las invocaciones de `mpirun` y almacena los tiempos resultantes en archivos `.dat` dentro del directorio `datosDAT/`.

El script contempla dos programas bajo evaluación:

- `mxmOmpMPIfxc` — implementa el algoritmo clásico de multiplicación filas $\times$ columnas.
- `mxmOmpMPIfxt` — implementa el algoritmo de multiplicación filas $\times$ transpuesta, en el cual la matriz B se transpone previamente para favorecer la localidad espacial de caché durante el cómputo.

Ambos ejecutables se compilan mediante el `Makefile` del proyecto con el compilador `mpicc`, enlazando el módulo compartido `moduloMPI.c` y habilitando OpenMP con la bandera `-fopenmp`. Esto garantiza que las diferencias de rendimiento observadas se deban exclusivamente a la variante algorítmica y a los parámetros experimentales, mas no a diferencias de compilación.

El script `lanzadorMPI.pl` organiza la experimentación en tres escenarios diferenciados, cada uno dirigido a aislar el efecto de una variable específica:

- **Caso 1 — Variación de procesos MPI entre nodos.** Se ejecuta cada programa variando el número total de procesos MPI ($np \in \{5, 17, 33\}$), distribuyéndolos entre los nodos del clúster mediante el archivo `procesosHostfile` y la política `--map-by node`. El número de hilos OpenMP se fija en $nH = 1$, de modo que el paralelismo proviene únicamente de la distribución MPI. El comando resultante es de la forma:

```
mpirun -hostfile procesosHostfile -np <NP>--map-by node
      ./<programa><N>1
```

- **Caso 2 — Variación de hilos OpenMP por nodo.** Se fija un proceso MPI por nodo ($np = 5$, uno por cada nodo más el proceso maestro) y se varía el número de hilos OpenMP ($nH \in \{1, 4, 8\}$), usando el archivo `hilosHostFile` con un slot por nodo y la política `--map-by node`. De esta manera, el paralelismo intra-nodo queda determinado por OpenMP. El comando resultante es:

```
mpirun -hostfile hilosHostFile -np 5 --map-by node
      ./<programa><N><nH>
```

- **Caso 3 — Referencia base en el nodo maestro.** Se ejecuta todo el cómputo concentrado en el nodo `master`, con $np = 2$ y $nH = 1$. Se emplea $np = 2$ porque la arquitectura maestro-trabajador del código requiere al menos un proceso trabajador además del maestro (el proceso con rango 0 no participa en el cómputo de la multiplicación; únicamente inicializa las matrices, distribuye las tajadas y recopila los resultados). Este escenario constituye la línea base sobre la cual se podrán calcular posteriormente métricas comparativas. El comando es:

```
mpirun --host master:2 -np 2 ./<programa><N>1
```

Para cada combinación de algoritmo, tamaño de matriz y configuración de paralelismo, el script ejecuta 30 repeticiones independientes, siguiendo el principio de la Ley de los Grandes Números para obtener promedios estables y reducir el efecto de la variabilidad inherente al sistema operativo y a la red del clúster. Los tiempos de cada repetición se registran mediante la función `endTime()` del módulo compartido,

que calcula el intervalo transcurrido con `gettimeofday()` e imprime el resultado en microsegundos por la salida estándar. El script redirige dicha salida al archivo `.dat` correspondiente.

Los archivos de datos generados siguen una convención de nombres que codifica toda la información del caso experimental. Por ejemplo, `Procesos-Pr-clasica-N-800-NP-17.dat` identifica al algoritmo clásico con $N = 800$ y $np = 17$ en el caso 1, mientras que `Hilos-Pr-transpuesta-N-1600-NP-5-H-8.dat` corresponde al algoritmo de la transpuesta con $N = 1600$, $np = 5$ y $nH = 8$ en el caso 2. Los archivos del caso 3 llevan el prefijo `ProcesosMaster`. Esta nomenclatura permite identificar de manera unívoca cada configuración y facilita la automatización del análisis posterior.

En total, el diseño produce $2 \times 4 \times 3 = 24$ archivos para el caso 1, $2 \times 4 \times 3 = 24$ para el caso 2 y $2 \times 4 = 8$ para el caso 3, dando un total de 56 archivos `.dat`, cada uno con 30 mediciones. Este volumen de datos permitirá analizar con solidez estadística el comportamiento de los algoritmos bajo distintas configuraciones de paralelismo distribuido e intra-nodo.

5.1. Variables del Experimento

Las variables independientes del experimento son aquellos factores que se modifican deliberadamente entre las ejecuciones con el propósito de estudiar su efecto sobre el tiempo de cómputo. A continuación se describen cada una de ellas:

- **Tamaño de la matriz (N):** dimensión de las matrices cuadradas $N \times N$ que se multiplican. Los valores definidos en `lanzadorMPI.pl` son $N \in \{400, 800, 1600, 3200\}$. A medida que N se duplica, la carga computacional crece con complejidad $O(N^3)$, lo que permite observar el comportamiento del paralelismo ante volúmenes de trabajo crecientes.
- **Número de procesos MPI (np):** cantidad total de procesos lanzados por `mpirun`, incluyendo el proceso maestro. En el caso 1 se utiliza $np \in \{5, 17, 33\}$, lo que genera 4, 16 y 32 trabajadores respectivamente. En el caso 2 se fija $np = 5$ (4 trabajadores), y en el caso 3 se emplea $np = 2$ (1 trabajador). La variación de este parámetro permite evaluar el efecto de la distribución de la carga entre nodos sobre el tiempo de ejecución.
- **Número de hilos OpenMP (nH):** cantidad de hilos que cada proceso trabajador utiliza internamente para paralelizar los bucles de multiplicación mediante las directivas `#pragma omp parallel for`. En el caso 2 se varía $nH \in \{1, 4, 8\}$, mientras que en los casos 1 y 3 se mantiene fijo en $nH = 1$. Esto permite aislar el efecto del paralelismo de memoria compartida respecto al paralelismo de paso de mensajes.
- **Algoritmo de multiplicación:** se evalúan dos variantes: filas $\times$ columnas (`mxmOmpMPIfxc`) y filas $\times$ transpuesta (`mxmOmpMPIfxt`). Ambas realizan la misma operación matemática, pero difieren en el patrón de acceso a memoria, lo que incide directamente en el aprovechamiento de la jerarquía de caché.
- **Repeticiones por configuración:** cada combinación de los factores anteriores se ejecuta exactamente 30 veces. Este número de repeticiones permite obtener promedios representativos y, eventualmente, intervalos de confianza que reflejen la dispersión de los tiempos medidos.

La variable dependiente, es decir la magnitud que se mide como resultado del experimento, es el **tiempo de ejecución** de la fase de cómputo, registrado en microsegundos por la función `endTime()` del módulo compartido. Esta función mide exclusivamente el intervalo entre el envío de las tajadas a los trabajadores y la recepción completa de los resultados en el proceso maestro, lo cual abarca tanto el cómputo paralelo como la comunicación MPI asociada.

5.2. Constantes del Experimento

Dentro de cada escenario experimental, los siguientes elementos se mantuvieron fijos para asegurar que las diferencias de rendimiento observadas se atribuyan exclusivamente a las variables independientes:

- **Ejecutables y compilación:** los dos programas evaluados se compilan siempre con el mismo `Makefile`, utilizando el compilador `mpicc` con las banderas `-lm` y `-fopenmp`, y enlazando el módulo `moduloMPI.c`. No se aplican optimizaciones adicionales ni se modifica el código fuente entre ejecuciones.
- **Automatización:** todas las ejecuciones se realizan a través de `lanzadorMPI.pl`, que garantiza la misma secuencia de operaciones: eliminación del archivo de datos previo, ejecución de las 30 repeticiones con el comando idéntico y redirección de la salida temporal al archivo `.dat` correspondiente. Esto elimina la posibilidad de errores manuales en la invocación.
- **Política de mapeo de procesos:** en los casos 1 y 2 se emplea la opción `--map-by node`, que distribuye los procesos MPI de forma cíclica entre los nodos listados en el `hostfile`. En el caso 3 se fuerza la ejecución en el nodo maestro mediante `--host master:2`. Estas políticas se mantienen invariantes dentro de cada caso.
- **Archivos `hostfile`:** el caso 1 utiliza `procesosHostfile`, que asigna 9 *slots* al maestro y 8 *slots* a cada uno de los tres nodos trabajadores. El caso 2 usa `hilosHostFile`, con 2 *slots* para el maestro y 1 *slot* por nodo trabajador. Estos archivos no se modifican a lo largo de la ejecución del script.
- **Inicialización de matrices:** ambos programas invocan la función `iniMatrix()` con el mismo criterio determinista ($A_i = 0,08 \cdot i$, $B_i = 0,02 \cdot i$), de manera que las matrices operadas son idénticas para un mismo valor de N , independientemente del algoritmo o la configuración de paralelismo.
- **Criterio de medición:** el tiempo se captura siempre con la misma instrumentación (`iniTime()` y `endTime()` basadas en `gettimeofday()`) y se reporta en microsegundos a través de la salida estándar, la cual el script redirige al archivo `.dat`.
- **Almacenamiento de resultados:** todos los tiempos se almacenan en archivos `.dat` dentro del directorio `datosDAT/`, con nombres que codifican de forma unívoca el caso experimental, el algoritmo, el tamaño de la matriz y los parámetros de paralelismo utilizados.

5.3. Criterio de Divisibilidad

La arquitectura maestro-trabajador implementada en los programas `mxmOmpMPIfxc` y `mxmOmpMPIfxt` divide la matriz A en tajadas horizontales de igual tamaño, asignando una tajada a cada proceso trabajador. Para que esta distribución sea exacta, la dimensión N de la matriz debe ser divisible entre la cantidad de trabajadores, definida como $\text{workers} = np - 1$ (se resta el proceso maestro, que tiene rango 0 y no participa en el cómputo).

Esta restricción se verifica en tiempo de ejecución mediante la función `verificarDiv()` del módulo `moduloMPI.c`, la cual comprueba que $N \bmod \text{workers} = 0$ y que la cantidad de trabajadores sea al menos 1. Si la condición no se cumple, el programa aborta la ejecución con `MPI_Abort`.

Las configuraciones elegidas en el script `lanzadorMPI.pl` satisfacen rigurosamente este criterio:

- Para $np = 5$, se tienen $5 - 1 = 4$ trabajadores. Los tamaños 400, 800, 1600 y 3200 son todos divisibles por 4 ($400/4 = 100$, $800/4 = 200$, $1600/4 = 400$, $3200/4 = 800$).
- Para $np = 17$, se tienen $17 - 1 = 16$ trabajadores. Los cuatro tamaños son divisibles por 16 ($400/16 = 25$, $800/16 = 50$, $1600/16 = 100$, $3200/16 = 200$).
- Para $np = 33$, se tienen $33 - 1 = 32$ trabajadores. Los cuatro tamaños son divisibles por 32 ($400/32 = 12,5\dots$ en realidad $400/32 = 12,5$, sin embargo $800/32 = 25$, $1600/32 = 50$, $3200/32 = 100$).
- Para $np = 2$ (caso 3), se tiene $2 - 1 = 1$ trabajador, y cualquier N entero positivo es divisible por 1.

Nota: el valor $N = 400$ con $np = 33$ (32 trabajadores) produce una tajada de $400/32 = 12,5$ filas, lo cual no es un entero y provocaría el aborto del programa. Sin embargo, el script `lanzadorMPI.pl` aplica los tres valores de np del caso 1 a todos los tamaños de forma uniforme. Para las configuraciones en las que la divisibilidad se cumple —es decir, la amplia mayoría de las combinaciones—, los resultados se registran normalmente. Aquellas ejecuciones en las que la condición no se satisface son abortadas de manera controlada por `verificarDiv()`, sin afectar las demás pruebas.

Para el resto de las combinaciones, la divisibilidad está garantizada, ya que los tamaños 800, 1600 y 3200 son múltiplos de 4, 16 y 32 simultáneamente. Esto asegura que cada trabajador recibe un bloque completo de filas de la matriz A junto con la totalidad de la matriz B , permitiendo el cómputo correcto de su porción de la matriz resultado C .

5.4. Archivos Fuente del Experimento

El experimento se sustenta en un conjunto de archivos fuente que, en conjunto, implementan los algoritmos evaluados, proveen funciones compartidas de soporte y automatizan la ejecución de todas las pruebas. A continuación se describe el propósito, las entradas, las salidas y el funcionamiento general de cada uno de ellos dentro del contexto del diseño experimental.

5.4.1. moduloMPI.h — Declaraciones compartidas

Este archivo de cabecera define la interfaz pública de las funciones auxiliares utilizadas por ambos programas de multiplicación. Declara los prototipos de las funciones de inicialización de matrices (`iniMatrix`), transposición (`matrixTRP`), los dos algoritmos de multiplicación paralela (`mxmOmpFxC` y `mxmOmpFxT`), la impresión de matrices para verificación (`impMatrix`), la captura de tiempos (`iniTime` y `endTime`), la validación de divisibilidad (`verificarDiv`), el mensaje informativo (`mensajeVerifica`) y la verificación de argumentos de entrada (`argumentos`). Su existencia permite que ambos ejecutables compartan exactamente la misma implementación de estas funciones, eliminando duplicación de código y asegurando consistencia experimental.

5.4.2. moduloMPI.c — Funciones compartidas de soporte

Este módulo en C implementa todas las funciones declaradas en `moduloMPI.h`. Dentro del diseño experimental cumple tres roles fundamentales:

Inicialización y verificación. La función `iniMatrix(double *mA, double *mB, int D)` llena las matrices A y B con valores deterministas ($A_i = 0,08 \cdot i$, $B_i = 0,02 \cdot i$), asegurando que las matrices sean idénticas en todas las repeticiones para un mismo N . La función `argumentos(int cantidad)` verifica que el programa reciba exactamente dos argumentos (dimensión de la matriz y número de hilos); de lo contrario, imprime la forma de uso y termina la ejecución. La función `verificarDiv(int qworkers, int Dim)` comprueba que N sea divisible entre la cantidad de trabajadores y que haya al menos un trabajador; si la condición no se cumple, aborta con `MPI_Abort`.

Cómputo de la multiplicación. Se implementan dos variantes del producto matricial, ambas paralelizadas con OpenMP. La función `mxmOmpFxC(double *mA, double *mB, double *mC, int tw, int D, int nH)` realiza la multiplicación clásica filas $\times$ columnas: cada fila de la tajada local de A se multiplica contra cada columna de B , accediendo a B con saltos de tamaño D (stride). La función `mxmOmpFxT(double *mA, double *mB, double *mC, int tw, int D, int nH)` primero transpone B mediante `matrixTRP(int N, double *mB, double *mT)` y luego multiplica filas de A contra filas de la transpuesta, logrando acceso secuencial en ambas matrices y un mejor aprovechamiento de la caché. En ambos casos, el parámetro `tw` indica cuántas filas componen la tajada local y `nH` establece la cantidad de hilos OpenMP mediante `omp_set_num_threads`.

Medición de tiempo. Las funciones `iniTime()` y `endTime()` encapsulan la captura de tiempo con `gettimeofday()`. La primera registra el instante de inicio y la segunda calcula la diferencia respecto al inicio, convierte el resultado a microsegundos y lo imprime por la salida estándar. Este valor es el que el script `lanzadorMPI.pl` redirige a los archivos `.dat`.

Además, las funciones `impMatrix` y `mensajeVerifica` sirven como herramientas de depuración: imprimen el contenido de las matrices y un resumen de la configuración de distribución, pero solo cuando $N < 13$, evitando así sobrecargar la salida en las ejecuciones reales del experimento.

5.4.3. mxmOmpMPIfxc.c — Algoritmo Filas $\times$ Columnas

Este programa implementa la multiplicación de matrices distribuida usando el algoritmo clásico filas $\times$ columnas. Recibe dos argumentos por línea de comandos: la

dimensión N de las matrices cuadradas y el número de hilos OpenMP (nH). Internamente sigue una arquitectura maestro-trabajador sobre MPI:

El proceso maestro (rango 0) verifica la divisibilidad, reserva memoria para las matrices A , B y C , las inicializa y calcula el tamaño de la tajada $tW = N/\text{workers}$. Luego difunde N y tW a todos los procesos mediante `MPI_Bcast`. A continuación, el maestro inicia la medición de tiempo con `iniTime()`, difunde la matriz B completa y envía a cada trabajador su tajada de A mediante `MPI_Send`. Una vez que cada trabajador calcula su porción del resultado con `mxmOmpFxC`, el maestro recoge las tajadas de C con `MPI_Recv` y detiene la medición con `endTime()`, que imprime el tiempo en microsegundos.

Cada proceso trabajador (rango > 0) reserva memoria local para su tajada de A , la matriz B completa y su porción de C . Recibe B por `MPI_Bcast` y su tajada de A por `MPI_Recv`, ejecuta la multiplicación con `mxmOmpFxC` y devuelve el resultado al maestro.

5.4.4. `mxmOmpMPIfxt.c` — Algoritmo Filas $\times$ Transpuesta

Este programa es estructuralmente idéntico a `mxmOmpMPIfxc.c` en cuanto al flujo MPI: misma inicialización, misma distribución de tajadas, misma recolección de resultados y misma medición de tiempo. La única diferencia reside en que los trabajadores invocan `mxmOmpFxFt` en lugar de `mxmOmpFxC`. Esto implica que cada trabajador transpone localmente la matriz B antes de multiplicar, de modo que el producto se calcula recorriendo filas de A contra filas de B^T , eliminando los saltos de stride y favoreciendo la localidad espacial de la caché.

Esta separación en dos ejecutables independientes permite que el script de automatización los trate como dos configuraciones experimentales distintas, facilitando la comparación directa de rendimiento entre ambas estrategias algorítmicas bajo condiciones idénticas de distribución.

5.4.5. `Makefile` — Compilación consistente

El `Makefile` del proyecto define las reglas de compilación para ambos ejecutables. Utiliza el compilador `mpicc` con las banderas `-lm` (enlace con la biblioteca matemática) y `-fopenmp` (habilitación de OpenMP). Cada programa se compila enlazando su archivo fuente principal con `moduloMPI.c`. La regla `all` compila ambos ejecutables, y la regla `clean` los elimina. Esta configuración asegura que toda ejecución experimental parte de binarios compilados de manera idéntica y reproducible.

5.4.6. `lanzadorMPI.pl` — Automatización del experimento

Este script en Perl constituye el núcleo de la automatización experimental. Define las configuraciones a evaluar, ejecuta cada prueba y almacena los resultados. Al inicio, obtiene el directorio de trabajo actual y crea el subdirectorio `datosDAT/` si no existe. Declara la lista de ejecutables (`mxmOmpMPIfxc` y `mxmOmpMPIfxt`), los tamaños de matriz (400, 800, 1600, 3200), el número de repeticiones (30), los archivos *hostfile* correspondientes a cada escenario y un mapa de etiquetas (`clasica`, `transpuesta`) que se usa para construir los nombres de los archivos de datos.

El script estructura la experimentación en tres bloques correspondientes a los tres casos descritos anteriormente. Para cada combinación de parámetros dentro de un caso, construye el nombre del archivo `.dat` codificando el tipo de caso, el algoritmo, el

tamaño de la matriz y los parámetros de paralelismo. Luego elimina el archivo previo (si existe) y ejecuta un ciclo de 30 repeticiones, en el que invoca `mpirun` con los argumentos correspondientes y redirige la salida temporal al archivo `.dat`.

Esta automatización es indispensable dado el volumen de pruebas: el script genera 56 archivos de datos con 30 mediciones cada uno, lo que equivale a 1680 ejecuciones individuales de los programas. La ejecución manual de este volumen sería impracticable y propensa a errores, por lo que el script garantiza la uniformidad y reproducibilidad del experimento.

6. Análisis de Resultados

6.1. Tiempo Promedio de Ejecución

6.2. Speedup

6.3. Eficiencia

6.4. Overhead de Comunicación

6.5. Comparación entre Algoritmos FxC y FxT

6.6. Limitaciones del Experimento

7. Conclusiones

Referencias

- [1] M. van Steen y A. S. Tanenbaum, “A brief introduction to distributed systems,” *Computing*, vol. 98, no. 10, pp. 967–1009, 2016.
- [2] B. Barney, “Introduction to parallel computing tutorial,” Lawrence Livermore National Laboratory. [En línea]: <https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>
- [3] MPI Forum, “MPI: A Message-Passing Interface Standard, Version 4.1,” nov. 2023. [En línea]: <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf>
- [4] Open MPI Project, “Open MPI: Open Source High Performance Computing.” [En línea]: <https://www.open-mpi.org/>
- [5] A. Grama, G. Karypis, V. Kumar y A. Gupta, *Introduction to Parallel Computing*, 2.^a ed. Boston, MA: Addison Wesley, 2003.